Best Decision Tree configurations and test results

dataset	criterion	max_depth	min_samples_split	min_samples_leaf	val acc	test acc	test F1
c300_d100	entropy	10	10	2	0.6700	0.6850	0.6850
c300_d1000	gini	5	2	1	0.6520	0.6725	0.6671
c300_d5000	gini	10	2	5	0.7404	0.7821	0.7818
c500_d100	entropy	5	2	5	0.6550	0.6800	0.6799
c500_d1000	gini	5	2	1	0.6940	0.6820	0.6819
c500_d5000	entropy	10	2	2	0.7587	0.7881	0.7876
c1000_d100	gini	10	2	2	0.7550	0.7250	0.7250
c1000_d1000	entropy	10	2	5	0.8100	0.7970	0.7967
c1000_d5000	entropy	10	10	2	0.8438	0.8609	0.8607
c1500_d100	entropy	10	2	5	0.8750	0.8800	0.8800
c1500_d1000	entropy	10	2	2	0.9165	0.9270	0.9270
c1500_d5000	entropy	20	10	2	0.9482	0.9565	0.9565
c1800_d100	gini	5	2	5	0.9800	0.9400	0.9400
c1800_d1000	entropy	10	2	2	0.9710	0.9755	0.9755

c1800_d5000	gini	10	2	2	0.9845	0.9875	0.9875

Reflection: I found that increasing max depth increased the model complexity and decreasing min_samples_split makes the tree more flexible. The best trees typically had moderate depth. Fully grown trees where the max depth was None didn't consistently win which means that when you don't constrain the height, the model tends to overfit. Also, a main note, is that the DT performance linearly increased as the # of clauses and dataset size increased.

Best Bagging configurations and test results

dataset	n_estimators	max_depth	val acc	test acc	test F1
c300_d100	100	5	0.7000	0.8550	0.8546
c300_d1000	100	10	0.8940	0.8885	0.8885
c300_d5000	100	None	0.9001	0.9185	0.9182
c500_d100	100	10	0.7850	0.8600	0.8600
c500_d1000	100	10	0.8795	0.8765	0.8765
c500_d5000	100	None	0.9134	0.9408	0.9408
c1000_d100	20	5	0.9050	0.8700	0.8699
c1000_d1000	100	None	0.9460	0.9475	0.9475
c1000_d5000	50	None	0.9556	0.9611	0.9611
c1500_d100	100	5	0.9750	0.9850	0.9850
c1500_d1000	100	10	0.9790	0.9825	0.9825
c1500_d5000	100	None	0.9891	0.9897	0.9897
c1800_d100	10	5	1.0000	0.9600	0.9600

c1800_d1000	100	10	0.9910	0.9925	0.9925
c1800_d5000	50	10	0.9964	0.9964	0.9964

Reflection: Bagging performed a lot better than DT on almost every dataset. Bagging reduces variance as it takes the average of several trees so this makes sense. The best number of estimators was mostly a 100, so larger ensembles helped. Deeper trees did not always perform the best as sometimes 5 or 10 was preferred. This suggests that the interaction between ensemble size and base learner complexity matters as bagging can support deeper trees better than a single, but deeper is not always the best.

Best Random Forest configurations and test results

dataset	n_estimators	max_features	val acc	test acc	test F1
c300_d100	100	0.5	0.7200	0.8250	0.8243
c300_d1000	100	0.5	0.8625	0.8875	0.8875
c300_d5000	100	0.5	0.8982	0.9249	0.9248
c500_d100	100	sqrt	0.8050	0.8700	0.8700
c500_d1000	100	sqrt	0.9050	0.9195	0.9195
c500_d5000	100	sqrt	0.9337	0.9471	0.9471
c1000_d100	100	log2	0.9700	0.9750	0.9750
c1000_d1000	100	sqrt	0.9940	0.9875	0.9875
c1000_d5000	100	log2	0.9950	0.9961	0.9961
c1500_d100	50	sqrt	1.0000	1.0000	1.0000
c1500_d1000	50	sqrt	1.0000	0.9995	0.9995

c1500_d5000	100	log2	1.0000	1.0000	1.0000
c1800_d100	20	log2	1.0000	1.0000	1.0000
c1800_d1000	50	sqrt	1.0000	1.0000	1.0000
c1800_d5000	20	log2	1.0000	0.9999	0.9999

Reflection: Random Forest also performed very well especially on the larger clause datasets as there is more training data. Increasing the number of estimators typically helped. Regarding the max_features setting -> log2 and sqrt were used most often while 0.5 worked better on the smaller clause datasets. The trade off between feature restriction and model strength is that feature restriction improves the diversity of the trees typically but too much restriction can also weaken the trees.

Best AdaBoost configurations and test results

dataset	n_estimators	base_depth	val acc	test acc	test F1
c300_d100	100	1	0.7300	0.7750	0.7745
c300_d1000	100	5	0.9125	0.9335	0.9335
c300_d5000	100	5	0.9752	0.9813	0.9813
c500_d100	100	5	0.8300	0.8850	0.8848
c500_d1000	100	5	0.9495	0.9735	0.9735
c500_d5000	100	5	0.9838	0.9914	0.9914
c1000_d100	100	1	0.9550	0.9750	0.9750
c1000_d1000	100	5	0.9910	0.9910	0.9910
c1000_d5000	100	5	0.9982	0.9991	0.9991
c1500_d100	50	3	0.9950	1.0000	1.0000

c1500_d1000	100	5	1.0000	1.0000	1.0000
c1500_d5000	100	5	1.0000	1.0000	1.0000
c1800_d100	50	3	1.0000	0.9950	0.9950
c1800_d1000	50	3	1.0000	0.9995	0.9995
c1800_d5000	50	3	1.0000	1.0000	1.0000

Reflection: AdaBoost had the strongest results in these experiments. The best n_estimators was typically a 100. Base depth 5 was the most common while 1 worked on the smaller settings and 3 worked at the higher end. So, increasing the # of estimators typically improved performance but didn't guarantee it. Same with deeper base learners in that it wasn't always the best but it worked often. This shows that stronger weak learners were useful but too much capacity could mean overfitting.

Comparative Analysis

Table 1. Test Accuracy

dataset	DecisionTree	Bagging	RandomForest	AdaBoost
c300_d100	0.6850	0.8550	0.8250	0.7750
c300_d1000	0.6725	0.8885	0.8875	0.9335
c300_d5000	0.7821	0.9185	0.9249	0.9813
c500_d100	0.6800	0.8600	0.8700	0.8850
c500_d1000	0.6820	0.8765	0.9195	0.9735
c500_d5000	0.7881	0.9408	0.9471	0.9914
c1000_d100	0.7250	0.8700	0.9750	0.9750
c1000_d1000	0.7970	0.9475	0.9875	0.9910

c1000_d5000	0.8609	0.9611	0.9961	0.9991
c1500_d100	0.8800	0.9850	1.0000	1.0000
c1500_d1000	0.9270	0.9825	0.9995	1.0000
c1500_d5000	0.9565	0.9897	1.0000	1.0000
c1800_d100	0.9400	0.9600	1.0000	0.9950
c1800_d1000	0.9755	0.9925	1.0000	0.9995
c1800_d5000	0.9875	0.9964	0.9999	1.0000

Table 2. Test Weighted F1

dataset	DecisionTree	Bagging	RandomForest	AdaBoost
c300_d100	0.6850	0.8546	0.8243	0.7745
c300_d1000	0.6671	0.8885	0.8875	0.9335
c300_d5000	0.7818	0.9182	0.9248	0.9813
c500_d100	0.6799	0.8600	0.8700	0.8848
c500_d1000	0.6819	0.8765	0.9195	0.9735
c500_d5000	0.7876	0.9408	0.9471	0.9914
c1000_d100	0.7250	0.8699	0.9750	0.9750
c1000_d1000	0.7967	0.9475	0.9875	0.9910
c1000_d5000	0.8607	0.9611	0.9961	0.9991
c1500_d100	0.8800	0.9850	1.0000	1.0000

c1500_d1000	0.9270	0.9825	0.9995	1.0000
c1500_d5000	0.9565	0.9897	1.0000	1.0000
c1800_d100	0.9400	0.9600	1.0000	0.9950
c1800_d1000	0.9755	0.9925	1.0000	0.9995
c1800_d5000	0.9875	0.9964	0.9999	1.0000

Question 1: Which classifier achieved the best overall generalization performance? Was the ranking consistent?

From my findings, AdaBoost proved to be the best in overall generalization performance. Its average test accuracy across all 15 datasets was about **0.9666**, compared with **0.9555** for Random Forest, **0.9349** for Bagging, and **0.8226** for Decision Tree. Weighted F1 followed the same pattern. The ranking wasn't perfectly consistent on every data set but the trend was **AdaBoost ≈ Random Forest > Bagging > Decision Tree.**

Question 2: How did increasing training data size affect performance? Which models benefited most?

Increasing the training data size typically improved the performance for all 4 classifiers.

Average test accuracy by data size (100 -> 1000 -> 5000):

- **Decision Tree:** 0.7820 → 0.8108 → 0.8750
- **Bagging:** 0.9060 → 0.9375 → 0.9613
- **Random Forest:** 0.9340 → 0.9588 → 0.9736
- **AdaBoost:** 0.9260 → 0.9795 → 0.9944

I can say that they all improved quite noticeably from the increase in test data but Random Forest and AdaBoost achieved near perfect results from the jump as did Bagging. DT was the weakest but still improved a lot from the data increase.

Question 3: How did increasing the number of clauses affect performance? What does that suggest?

Increasing the number of clauses generally made the performance better. For example, average Decision Tree accuracy rose from about **0.7132** at c300 to about **0.9677** at c1800. This suggests that the larger-clause formulas could have created a classification boundary that was easier for tree-based methods to separate.